Full system simulation in gem5



What we will cover

- What is full system simulation?
- Basics of booting up a real system in gem5
- Creating disk images using Packer and QEMU
- Extending/modifying a gem5 disk image
- Using m5term to interact with a running system

What is Full System Simulation?

Full-system simulation is a type of simulation that emulates a complete computer system, including the CPU, memory, I/O devices, and system software like operating systems.

It allows for detailed analysis and debugging of hardware and software interactions.

Components Simulated:

- CPUs (multiple types and configurations)
- Memory hierarchy (caches, main memory)
- I/O devices (disk, network interfaces)
- Entire software stack (OS, drivers, applications)

Basics of Booting Up a Real System in gem5

Overview: gem5 can simulate the process of booting up a real system, providing insights into the behavior of the hardware and software during startup.

Steps Involved

1. **Setting Up the Simulation Environment**:
   - Choose the ISA (e.g., x86, ARM).
   - Configure the system components (CPU, memory, caches).
2. **Getting the correct resources such as kernel, bootloader, diskimages, etc.**
3. **Configuring the Boot Parameters**:
   - Set kernel command line parameters, if necessary.
4. **Running the Simulation**:
   - Start the simulation and monitor the boot process.

Let's run a full system simulation in gem5

The incomplete code already has a board built.

Let's run a full-system workload in gem5.

This workload is an Ubuntu 24.04 boot. It will throw three m5 exits at:

- Kernel Booted
- When `after_boot.sh` runs
- After run script runs

Obtain the workload and set exit event

To set the workload, we add the following to materials/02-Using-gem5/07-full-system/x86-fs-kvm-run.py:

```
workload = obtain_resource("x86-ubuntu-24.04-boot-with-systemd", resource_version="1.0.0")
board.set_workload(workload)
```

Obtain the workload and set exit event (conti.)

Let's make the exit event handler and set it in our simulator's object.

```
def exit_event_handler():
    print("first exit event: Kernel booted")
    yield False
    print("second exit event: In after boot")
    yield False
    print("third exit event: After run script")
    yield True

simulator = Simulator(
    board=board,
    on_exit_event={
        ExitEvent.EXIT: exit_event_handler(),
    },
)
simulator.run()
```

Viewing the terminal/serial output with m5term

Before booting this workload, let's build the `m5term` application so we can connect to the running system.

```
cd /workspaces/2024/gem5/util/term
make
```

Now you have a binary `m5term`.

Watch gem5's output

Now, let's run the workload and connect to the terminal of the disk image boot using `m5term`.

Run gem5 with:

```
gem5 x86-fs-kvm-run.py
```

In another terminal window, run the following command to connect to the disk image boot's terminal:

```
m5term 3456
```

3456 is the port number on which the terminal is running.
You will see this printed in the gem5 output.

If you run multiple gem5 instances, they will have sequential port numbers.
If you are running in a non-interactive environment, there will be no ports to connect to.



Creating your own disk images



Creating disk images using Packer and QEMU

To create a generic Ubuntu disk image that we can use in gem5, we will use:

- Packer: This will automate the disk image creation process.
- QEMU: We will use a QEMU plugin in Packer to actually create the disk image.
- Ubuntu autoinstall: We will use autoinstall to automate the Ubuntu install process.

gem5 resources already has code that can create a generic Ubuntu image using the aforementioned method.

- Path to code: `gem5-resources/src/x86-ubuntu`

Let's go through the important parts of the creation process.

Getting the ISO and the user-data file

As we are using Ubuntu autoinstall, we need a live server install ISO.

- This can be found online from the Ubuntu website: iso

We also need the user-data file that will tell Ubuntu autoinstall how to install Ubuntu.

- The user-data file on gem5-resources specifies all default options with a minimal server installation.

How to get our own user-data file

To get a user-data file from scratch, you need to install Ubuntu on a machine.

- Post-installation, we can retrieve the `autoinstall-user-data` from `/var/log/installer/autoinstall-user-data` after the system's first reboot.

You can install Ubuntu on your own VM and get the user-data file.

Using QEMU to get the user-data file

We can also use QEMU to install Ubuntu and get the aforementioned file.

- First, we need to create an empty disk image in QEMU with the command: `qemu-img create -f raw ubuntu-22.04.2.raw 5G`
- Then we use QEMU to boot the diskimage:

```
qemu-system-x86_64 -m 2G \
      -cdrom ubuntu-22.04.2-live-server-amd64.iso \
      -boot d -drive file=ubuntu-22.04.2.raw,format=raw \
      -enable-kvm -cpu host -smp 2 -net nic \
      -net user,hostfwd=tcp::2222-:22
```

After installing Ubuntu, we can use ssh to get the user-data file.

Important parts of the Packer script

Let's go over the Packer file.

- **bootcommand**:

```
"e<wait>",
"<down><down><down>",
"<end><bs><bs><bs><bs><wait>",
"autoinstall  ds=nocloud-net\\;s=http://{{ .HTTPIP }}:{{ .HTTPPort }}/ ---<wait>",
"<f10><wait>"
```

  This boot command opens the GRUB menu to edit the boot command, then removes the `---` and adds the autoinstall command.

- **http_directory**: This directory points to the directory with the user-data file and an empty file named meta-data. These files are used to install Ubuntu.



Important parts of the Packer script (Conti.)

- **qemu_args**: We need to provide Packer with the QEMU arguments we will be using to boot the image.
  - For example, the QEMU command that the Packer script will use will be:

```
qemu-system-x86_64 -vnc 127.0.0.1:32 -m 8192M \
-device virtio-net,netdev=user.0 -cpu host \
-display none -boot c -smp 4 \
-drive file=<Path/to/image>,cache=writeback,discard=ignore,format=raw \
-machine type=pc,accel=kvm -netdev user,id=user.0,hostfwd=tcp::3873-:22
```

- **File provisioners**: These commands allow us to move files from the host machine to the QEMU image.
- **Shell provisioner**: This allows us to run bash scripts that can run the post installation commands.



Let's use the base Ubuntu image to create a disk image with the GAPBS benchmarks

Update the x86-ubuntu.pkr.hcl file.

The general structure of the Packer file would be the same but with a few key changes:

- We will now add an argument in the `source "qemu" "initialize"` block.
  - `diskimage = true` : This will let Packer know that we are using a base disk image and not an iso from which we will install Ubuntu.
- Remove the `http_directory = "http"` directory as we no longer need to use autoinstall.
- Change the `iso_checksum` and `iso_urls` to that of our base image.

  Let's get the base Ubuntu 24.04 image from gem5 resources and unzip it.

```
wget https://dist.gem5.org/dist/develop/images/x86/ubuntu-24-04/x86-ubuntu-24-04.gz
gzip -d x86-ubuntu-24-04.gz
```

`iso_checksum` is the `sha256sum` of the iso file that we are using. To get the `sha256sum` run the following in the linux terminal.

```
sha256sum ./x86-ubuntu-24-04.gz
```

- **Update the file and shell provisioners:** Let's remove the file provisioners as we dont need to transfer the files again.
- **Boot command:** As we are not installing Ubuntu, we can write the commands to login along with any other commands we need (e.g. setting up network or ssh). Let's update the boot command to login and enable network:

```
"<wait30>",
"gem5<enter><wait>",
"12345<enter><wait>",
"sudo mv /etc/netplan/50-cloud-init.yaml.bak /etc/netplan/50-cloud-init.yaml<enter><wait>",
"12345<enter><wait>",
"sudo netplan apply<enter><wait>",
"<wait>"
```

Changes to the post installation script

For this post installation script we need to get the dependencies and build the GAPBS benchmarks.

Add this to the post-installation.sh script

```
git clone https://github.com/sbeamer/gapbs
cd gapbs
make
```

Let's run the Packer script and use this disk image in gem5!

```
cd /workspaces/2024/materials/02-Using-gem5/07-full-system
x86-ubuntu-gapbs/build.sh
```

Let's use our built disk image in gem5

Let's add the md5sum and the path to our local JSON .

Let's run the gem5 GAPBS config.

```
GEM5_RESOURCE_JSON_APPEND=./completed/local-gapbs-resource.json gem5 x86-fs-gapbs-kvm-run.py
```

This script should run the bfs benchmark.

Let's see how we can access the terminal using m5term

- We are going to run the same gem5 GAPBS config but with a small change.

Let's change the last `yield True` to `yield False` so that the simulation doesn't exit and we can access the simulation.

```
def exit_event_handler():
    print("first exit event: Kernel booted")
    yield False
    print("second exit event: In after boot")
    yield False
    print("third exit event: After run script")
    yield False
```

Again, let's use m5term

Now let's connect to our simulation by using the `m5term` binary

```
m5term 3456
```